

PANDAS

Start coding or [generate](#) with AI.

```
!pip install pandas
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.0.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2026.1)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.
```

```
import pandas as pd
import numpy as np
```

```
print(pd.__version__)
```

```
2.2.2
```

```
#SERIES
s=pd.Series([1,2,3,4])
print(s)
```

```
0    1
1    2
2    3
3    4
dtype: int64
```

```
#from list
s=pd.Series([1,2,3,4], index=['a','b','c','d'])
print(s)
```

```
a    1
b    2
c    3
d    4
dtype: int64
```

```
#dict

stu={
    "name":"aswi",
    "branch":"cse"
}
se=pd.Series(stu)
print(stu)
```

```
{'name': 'aswi', 'branch': 'cse'}
```

```
#
p=pd.Series([1,2,3,4,5,5,6,7,8])
```

```
#airthmetic
p*3
```

```
0
0 3
1 6
2 9
3 12
4 15
5 15
6 18
7 21
8 24
```

dtype: int64

p+50

```
0
0 51
1 52
2 53
3 54
4 55
5 55
6 56
7 57
8 58
```

dtype: int64

```
#statistics
s.mean()
```

np.float64(2.5)

s.sum()

np.int64(10)

s.max()

4

```
#vetroized string operations
names=pd.Series(['vamsi','rani'])
```

names.str.lower()

```
0
0 vamsi
1 rani
```

dtype: object

names.str.len()

```
0
```

```
0 5
```

```
1 4
```

```
dtype: int64
```

```
names.str.startswith()
```

```
-----
AttributeError                                Traceback (most recent call last)
/tmp/ipykernel_1290/174060112.py in <cell line: 0>()
----> 1 names.str.startswith()

AttributeError: 'StringMethods' object has no attribute 'startswith'
```

```
stu={
    "names":["van","man","lan"],
    "branch":["cse","ece","eee"]
}
df=pd.DataFrame(stu)
print(df)
```

```
names branch
0  van    cse
1  man    ece
2  lan    eee
```

```
#feom list dict
std=[
    {"name":"vamsi","branch":"cse","age":20},
    {"name":"man","branch":"ece"},
    {"name":"lan","branch":"eee"},
]
df=pd.DataFrame(std)
print(df)
```

```
name branch age
0  vamsi    cse  20.0
1   man    ece   NaN
2   lan    eee   NaN
```

```
# from numpy array
arr=np.random.rand(3,4)
c=pd.DataFrame(arr,columns=['a','b','c','d'])
print(c)
```

```
      a      b      c      d
0  0.831204  0.139533  0.282404  0.477307
1  0.119162  0.876430  0.881445  0.507587
2  0.264641  0.119955  0.849984  0.615987
```

```
data={
    'name':['vamsi','man','lan'],
    'branch':['cse','ece','eee'],
    'year':['2','3','4'],
    'grade':['a','b','c'],
    'age':['20','21','22']
}
df=pd.DataFrame(data)
print(df)
```

```
name branch year grade age
0  vamsi    cse   2     a  20
1   man    ece   3     b  21
2   lan    eee   4     c  22
```

```
df.head()
```

```
name branch year grade age
0  vamsi    cse   2     a  20
1   man    ece   3     b  21
2   lan    eee   4     c  22
```

```
df.shape
```

```
(3, 5)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 5 columns):
#   Column  Non-Null Count  Dtype
---  -
0    name    3 non-null      object
1   branch  3 non-null      object
2    year    3 non-null      object
3   grade   3 non-null      object
4    age     3 non-null      object
dtypes: object(5)
memory usage: 252.0+ bytes
```

```
df.describe()
```

	name	branch	year	grade	age
count	3	3	3	3	3
unique	3	3	3	3	3
top	vamsi	cse	2	a	20
freq	1	1	1	1	1

```
df.columns
```

```
Index(['name', 'branch', 'year', 'grade', 'age'], dtype='object')
```

```
df.index
```

```
RangeIndex(start=0, stop=3, step=1)
```

```
df.dtypes
```

```

0
name    object
branch  object
year    object
grade   object
age     object

dtype: object
```

```
# Separate them like this:
df['name']
```

```

name
0  vamsi
1   man
2   lan

dtype: object
```

```
df.name
```

```

name
0  vamsi
1   man
2   lan

dtype: object
```

```
df[['name', 'age']]
```

	name	age
0	vamsi	20
1	man	21
2	lan	22

```
df.loc[0]
```

	0
name	vamsi
branch	cse
year	2
grade	a
age	20

dtype: object

```
df.loc[0:2]
```

	name	branch	year	grade	age
0	vamsi	cse	2	a	20
1	man	ece	3	b	21
2	lan	eee	4	c	22

```
df.loc[0:2, ['name', 'age']]
```

	name	age
0	vamsi	20
1	man	21
2	lan	22

```
df.iloc[0:2,0:2]
```

	name	branch
0	vamsi	cse
1	man	ece

```
df.iloc[-1]
```

	2
name	lan
branch	eee
year	4
grade	c
age	22

dtype: object

```
df.iloc[0:2]
```

	name	branch	year	grade	age
0	vamsi	cse	2	a	20
1	man	ece	3	b	21

```
df.iloc[0]
```

```

0
name    vamsi
branch   cse
year      2
grade     a
age      20
dtype: object
```

```
df.iloc[0:0]
```

	name	branch	year	grade	age
--	------	--------	------	-------	-----

```
df.loc[0, 'name']
```

```
'vamsi'
```

Afternoon session

```
df[(df['age'] < '20') & (df['branch'] == 'a')]
```

	name	branch	year	grade	age
--	------	--------	------	-------	-----

```

data={
    'name':['anu', 'pooji', 'lucky'],
    'age':[20,21,22],
    'score':[np.nan,80,70],
    'grade':['A', 'B', 'C']
}
d=pd.DataFrame(data)
print(d)
```

	name	age	score	grade
0	anu	20	NaN	A
1	pooji	21	80.0	B
2	lucky	22	70.0	C

```

d['example']=["a","b","c"]
d['grade']=['pass','fail','pass']
```

```

d['grade']=d['score']>=80
print(d)
```

	name	age	score	grade	example
0	anu	20	90	True	a
1	pooji	21	80	True	b
2	lucky	22	70	False	c

```

#modify existing column
d['score']=d['score']+10
print(d)
```

	name	age	score	grade	example
0	anu	20	100	True	a
1	pooji	21	90	True	b
2	lucky	22	80	False	c

```
def grade(score):
    if score>=90:
        return "A"
    elif score>=80:
        return "B"
    else:
        return "C"
d[grade]=d['score'].apply(grade)
print(d)
```

```
-----
TypeError                                Traceback (most recent call last)
/tmp/ipykernel_1290/1283974312.py in <cell line: 0>()
      6     else:
      7         return "C"
----> 8 d[grade]=d['score'].apply(grade)
      9 print(d)

----- 10 frames -----
/usr/local/lib/python3.12/dist-packages/pandas/core/ops/array_ops.py in comp_method_OBJECT_ARRAY(op, x, y)
    127     result = libops.vec_compare(x.ravel(), y.ravel(), op)
    128     else:
--> 129     result = libops.scalar_compare(x.ravel(), y, op)
    130     return result.reshape(x.shape)
    131

ops.pyx in pandas._libs.ops.scalar_compare()

TypeError: '>=' not supported between instances of 'str' and 'int'
```

```
result={
    True : "pass",
    False: "fail"
}
pas={'A':90,'B':80,'C':60}
d['pas']=d['Grade'].map(pas)
d['result']=d['grade'].map(result)
print(d)
```

```
-----
KeyError                                Traceback (most recent call last)
/usr/local/lib/python3.12/dist-packages/pandas/core/indexes/base.py in get_loc(self, key)
    3804         try:
-> 3805             return self._engine.get_loc(casted_key)
    3806         except KeyError as err:

index.pyx in pandas._libs.index.IndexEngine.get_loc()

index.pyx in pandas._libs.index.IndexEngine.get_loc()

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'Grade'
```

The above exception was the direct cause of the following exception:

```
-----
KeyError                                Traceback (most recent call last)
----- 2 frames -----
/usr/local/lib/python3.12/dist-packages/pandas/core/indexes/base.py in get_loc(self, key)
    3810         ):
    3811             raise InvalidIndexError(key)
-> 3812         raise KeyError(key) from err
    3813     except TypeError:
    3814         # If we have a listlike key, _check_indexing_error will raise

KeyError: 'Grade'
```

```
print(d)
```

	name	age	score
0	anu	20	NaN
1	pooji	21	80.0
2	lucky	22	70.0

```
d.drop('grade',axis=1,inplace=True)
```

```
print(d)
```

	name	age	score	grade
0	anu	20	NaN	A

```
1 pooji 21 80.0 B
2 lucky 22 70.0 C
```

```
d.replace('A',np.nan,inplace=True)
```

```
d.sort_values(['age','score'],ascending=[True,False])
```

	name	age	score	grade
0	anu	20	NaN	NaN
1	pooji	21	80.0	B
2	lucky	22	70.0	C

```
d.sort_index(ascending=False)
```

	name	age	score	grade
2	lucky	22	70.0	C
1	pooji	21	80.0	B
0	anu	20	NaN	NaN

```
d.nlargest(2,'score')
d.nsmallest(3,'age')
```

	name	age	score	grade
0	anu	20	NaN	NaN
1	pooji	21	80.0	B
2	lucky	22	70.0	C

```
df.groupby('departmwn')
```

```
employees = pd.DataFrame({
    'emp_id': [1, 2, 3, 4],
    'name': ['Alice', 'Bob', 'Charlie', 'Diana'],
    'dept id': [10, 20, 10, 30]
```